MCU – SoC – FPGA

Trade-offs in Performance, Cost and Flexibility

CO3053 Embedded Systems · Semester 261 · Topic 09

The question

When should you choose an **MCU**, an **FPGA** or a **SoC**?

- Judged on three criteria: **performance**, **cost** and **flexibility**
- *Not* an attempt to prove one platform is “the best”: `pe2` lists a biased goal as a common mistake
- Every number on these slides carries a tag saying where it comes from: [measured] [simulated] [tool estimate] [calculated] [source]

Roadmap

1. **MCU**: what it is, strengths, weaknesses, where it wins
2. **FPGA**: what it is, strengths, weaknesses, where it wins
3. **SoC**: what it is, strengths, weaknesses, where it wins
4. **Comparison**: published data, our test case, our results, decision matrix
5. Conclusion

Each platform is first examined on its own, **with the same template**: what it is → where it is strong → where it is weak → which domains it wins.

Only then are the three placed **side by side**, on the same axes and on the same FIR workload.

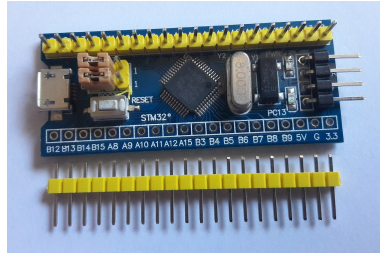
1

MCU

Microcontroller: small, cheap, frugal and on time

What is an MCU?

- One chip with a **small CPU core** (ARM Cortex-M), **flash**, **SRAM** and **peripherals**: timers, ADC, PWM, UART, SPI, I^2C
- Runs **bare-metal** or an **RTOS**, programmed in C with direct register access (Lecture 2)
- Executes **sequentially**: one thing per cycle
- **No MMU**, so it cannot run Linux
- Our board: Blue Pill STM32F103C6, Cortex-M3 at 72 MHz



Popolon, CC BY-SA 4.0

Blue Pill board (STM32F103 family)

MCU: strengths and weaknesses

Strengths

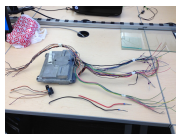
- **Cheap**: STM32H743 £9–12 per chip; the F103 family costs a few dollars [source]
- **Extremely frugal when idle**: STM32U5 300 nA, H743 2.43 µA in Standby [source]; 10+ years on a coin cell (TI)
- **On time**: bare-metal interrupt latency 1–3 µs [source: AC6]; we measured 264–347 ns [measured]
- **Boots in milliseconds**, ready as soon as power is applied
- **Safety-certified**: AURIX reaches ASIL-D; AEC-Q100 Grade 0 up to 150 °C
- **Fast time-to-market** for simple, fixed functions (Lecture 2)

Weaknesses

- **Low performance**: CoreMark 608–2,424 [source]
- **Not enough for parallel work**: H743 peaks at ≈0.48 G MAC/s, ≈29% of what the video filter needs [calculated]
- **No Linux**: TCP/IP, file system, camera drivers written by hand
- **Worst case is set by the software**: one higher-priority 20 µs ISR pushes latency to 21.9 µs [measured]
- The M3 has **no SIMD**: the fastest version still takes 216.3 cycles/sample; CMSIS-DSP is no faster than C [measured]

Where the MCU wins: automotive ECUs

- A mainstream car carries 25–35 MCUs, a luxury car 60–70
- AURIX TC3xx: 4 of 6 cores run in **lockstep**, targeting **ASIL-D** (ISO 26262), for powertrain and EV inverters
- AEC-Q100 Grade 0: operates from −**40 to 150 °C**; NXP S32K14xW comes with a 15-year supply commitment
- Boots in milliseconds, sub-µs interrupts, a few dollars per chip



Snewkirk7953, CC BY-SA 3.0

Why not a SoC: Linux boots slowly and its latency is not deterministic

Why not an FPGA: expensive; configuration lives in SRAM, so it is exposed to radiation upsets (SEU); hard to certify for millions of cars

Where the advantage ends: ADAS and infotainment use SoCs; domain controllers are merging many ECUs into fewer units

Also dominant in: FOC motor control (under 21 µs of a 50 µs budget), battery sensor nodes (300 nA, over 10 years on a CR2032), home appliances

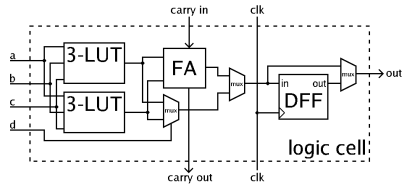
2

FPGA

Programmable logic: true parallelism, fixed latency

What is an FPGA?

- An array of thousands of **logic cells** (LUT + flip-flop), plus **DSP** blocks (multiply-accumulate) and **block RAM**, wired together freely
- It runs no program: the algorithm is **laid out in space**, each operation gets its own circuit and all of them run at once
- Described in an **HDL** (Verilog, VHDL); the configuration lives in SRAM, so it can be **reloaded**
- Our board: Arty Z7-20, the PL of an XC7Z020 (220 DSP48E1, 53,200 LUTs); our FIR runs **LUT-only, 0 DSPs**



Petter.kallstrom, Public domain

One logic cell: LUTs, adder, flip-flop



S03311251, CC BY-SA 4.0

FPGA: strengths and weaknesses

Strengths

- **True parallelism**: 220–240 DSPs give ≈44–48 G MAC/s at 200 MHz [calculated]; XC7A100T quoted up to 264 G [source]
- **One sample per clock**: our FIR reaches 1.00 samples/cycle [measured]
- **Fixed latency, in nanoseconds**: 13.9 ns tick-to-trade [source]; our pipeline takes 7 cycles = 56 ns
- **Low energy per operation**: FIR on an FPGA uses ≈95–190× less energy than on a CPU [calculated, Arucu & Iliev]
- **I/O wired straight to the pins**, and **reloadable** when a standard changes

Weaknesses

- **Expensive**: XC7A100T (Artix-7 family) €143.90 per chip [source]
- **Hard to program**: every operation must be described at hardware level (Arucu & Iliev)
- **Less flexible**: adding an FPGA lowers flexibility from “High” to “Medium” (Lecture 3)
- **No operating system**: a network stack or a user interface must be built from scratch
- **Does not win small problems**: one desktop CPU core filters a 16-tap FIR at ≈140 MSPS [measured, x86 reference]

Where the FPGA wins: data streams and chip prototyping

For parallel computing in general, GPUs are more common. The FPGA wins when data **streams from an ADC**, latency must be **fixed**, and the standard still changes or volume is too small for an ASIC.

- **Ultrasound:** beamforming 128–1,024 channels, 4.83 GS/s on one FPGA, wired to the analog front end
- **5G radio units:** low-PHY and DFE on RFSoC or Agilex; O-RAN still evolves, so reloading matters
- **ASIC prototyping:** HAPS runs up to 1.6 billion gates of RTL on many FPGAs before tape-out
- **HFT:** 13.9 ns tick-to-trade



Erereony, CC BY 4.0

Why not an MCU: ≈3.5× short of the needed throughput even at 100% efficiency

Why not a SoC: SIMD or a GPU may keep up, but determinism is lost and power rises

Where it ends: at high volume with a stable algorithm, an ASIC wins (a phone's ISP, modem)

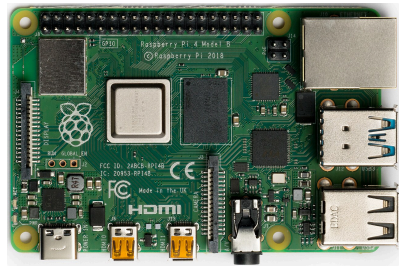
3

SoC

System on chip: strong CPU + Linux + hardware accelerators

What is a SoC?

- **Application** cores (ARM Cortex-A) with an **MMU** and **caches**, plus a GPU, video codecs and network accelerators, all on one chip
- Runs **Linux**: drivers, network stack and file system come ready-made (Lecture 2)
- Dedicated hardware blocks (packet engine, ISP, NPU, modem) do the heavy lifting; the CPU handles control
- Our board: Raspberry Pi 4, 4×Cortex-A72 at 1.8 GHz



Laserlicht, CC BY-SA 4.0

Raspberry Pi 4 Model B

SoC: strengths and weaknesses

Strengths

- **Strongest CPU of the three**: the Pi 4 scores 38,200 CoreMark, ≈16× the H743 [source]
- **Linux**: cameras work out of the box via V4L2, plus networking, file systems and remote updates
- **Integrated hardware blocks**: router packet engine at 37.5 Mpps; phones add an NPU, ISP and 5G modem [source]
- **Cheap boards**: Pi 4 launched at $35 [source]
- **Fast time-to-market** for feature-rich, connected products (Lecture 2)

Weaknesses

- **Non-deterministic latency**: PREEMPT_RT 30–80 µs, standard Linux a few ms [source: AC6]
- **Power-hungry even when idle**: Pi 4 ≈3 W for the whole board, ≈7 W at peak [source]
- **Slow to boot**, complex software stack (kernel, drivers, security)
- **Weak for hard real time** without a real-time patch (Lecture 2)

Where the SoC wins: Wi-Fi routers and phones

- **Router:** Qualcomm IPQ = CPU running Linux/OpenWrt + Wi-Fi + switch + **hardware packet engine** (37.5 Mpps) + crypto
- **Phone:** Snapdragon 8 Elite = CPU + GPU + NPU + ISP + 5G modem on a 3 nm chip
- Linux runs the control plane; hardware blocks run the data plane



Raimond Spekking, CC BY-SA 4.0



Raimond Spekking, CC BY-SA 4.0

Why not an MCU: no MMU, so no Linux; the whole TCP/IP stack would have to be rewritten

Why not an FPGA: writing a network stack and a UI in HDL is far too costly for a consumer device

Where the advantage ends: hard real-time control, which is why these devices still carry small MCUs

Also dominant in: IP cameras, TVs and set-top boxes, IoT gateways, in-car infotainment

When you need both: hybrid CPU + FPGA

- The problem needs **both** a rich operating system **and** hard timing or heavy parallel work (ADAS, industrial control)
- Zynq UltraScale+ puts on one chip: 4×A53 running Linux, 2×R5 in lockstep for real time, and FPGA fabric
- Lecture 3, Option #2: the CPU handles flexible computation, the FPGA keeps actuator timing even when the CPU is busy
- FIR on an FPGA+ARM SoC ran 8.86× faster than ARM alone, with the best price/performance of the configurations tested [source]



S03311251, CC BY-SA 4.0

Zynq UltraScale+: Arm + FPGA on one chip

The price: higher cost and lower flexibility than a single CPU

4

Comparison

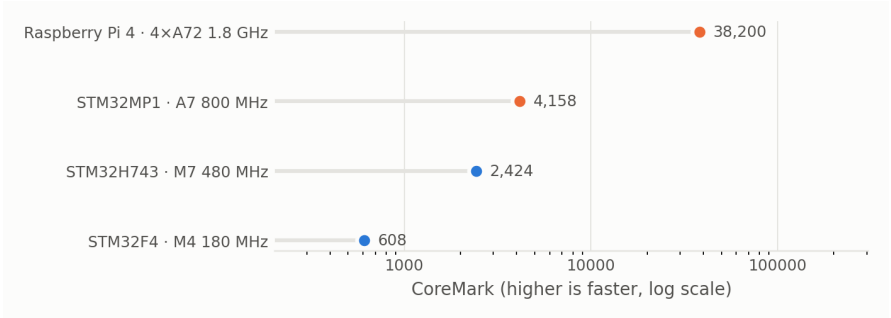
The three platforms on the same axes

Trade-off overview

Criterion	MCU	FPGA	SoC
Sequential CPU (CoreMark)	Low: 608–2,424	Not applicable	**High: 38,200** (Pi 4)
Parallel throughput	≈0.48 G MAC/s [calculated]	≈**48 G MAC/s** [calculated]	Medium (SIMD, GPU)
Worst-case latency	1–3 µs	**nanoseconds**	30 µs – a few ms
Idle power	**µW range** (chip)	no data	≈3 W (board)
Price	**£9–12/chip**	€144/chip	$35/board
Programming	C, registers	HDL, hardest	**Linux, rich libraries**
Flexibility / time-to-market	Fast for simple products	Lowest	Fast for feature-rich products

Sources: ST, OpenBenchmarking, AC6, STAC-T0, RS, Farnell, AnandTech, Lectures 2–3. Bold marks the winner for each criterion. The next slides look at each criterion.

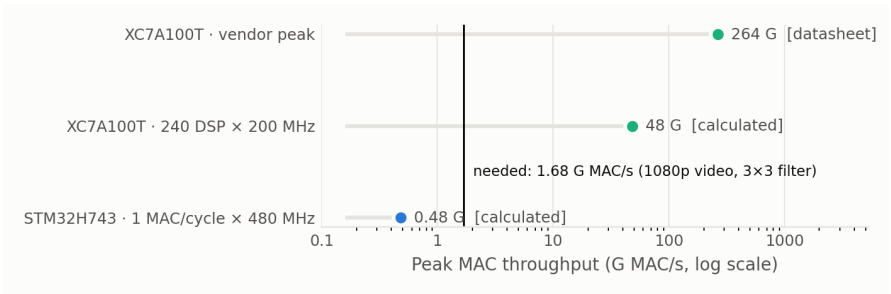
Raw CPU performance: CoreMark



Data: ST product overview; ST STM32H743; OpenBenchmarking (Pi 4, all 4 cores). Re-plotted by the team.

The **SoC** (Pi 4) has ≈16× the CPU performance of the **MCU** (H743). CoreMark does not apply to the **FPGA**, which runs no sequential program.

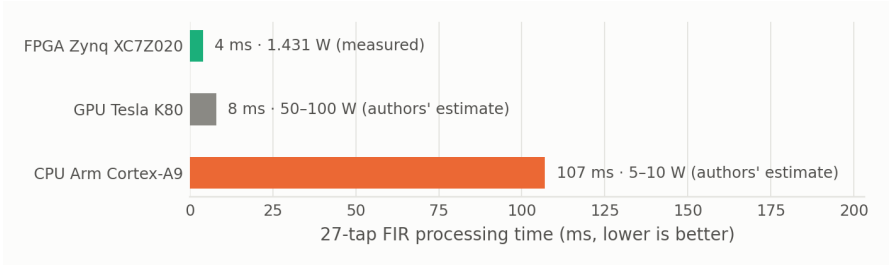
Parallel throughput: filtering 1080p video



Data: Lee & Seshia §8.1 (requirement); XA Artix-7 datasheet (264 G). [calculated] values follow the stated assumptions. Re-plotted by the team.

The **MCU** covers only ≈29% of the requirement at 100% efficiency; the **FPGA** uses only ≈3.5% of its capacity [calculated]

Published evidence: FIR on FPGA, GPU and CPU

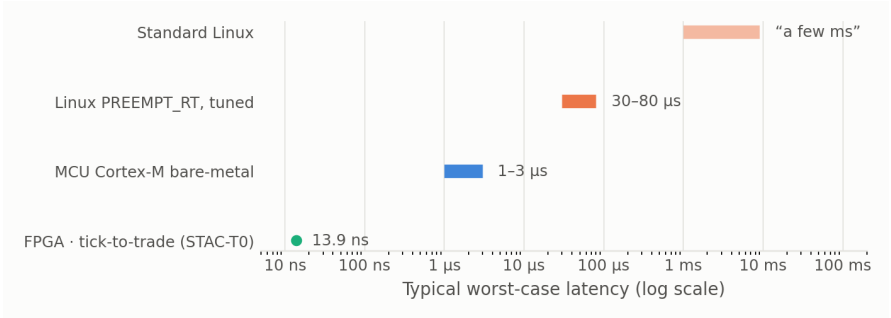


Data: Arucu & Iliev, J. Low Power Electron. Appl. 15(3):40, 2025. Re-plotted by the team.

The **FPGA** is ≈27× faster than a Cortex-A9 and uses ≈95–190× less energy [calculated].

CPU and GPU power are the authors' estimates; the study has no MCU.

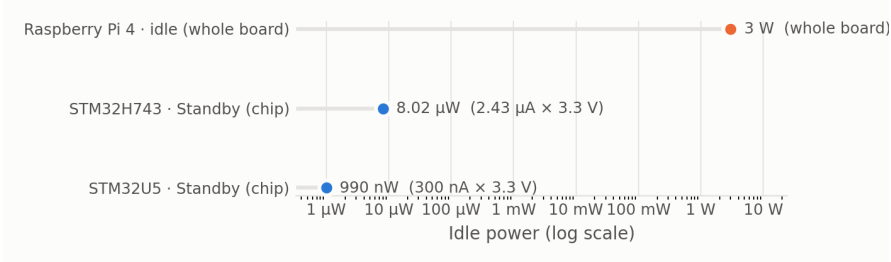
Worst-case latency in the literature



Data: AC6 (MCU, Linux); A-Team Insight, STAC-T0 (FPGA). “A few ms” has no exact figure, so it is drawn faded. Re-plotted by the team.

Each platform sits 1–2 orders of magnitude from the next. The measurements differ in nature, so compare orders of magnitude only.

Idle power and price



Data: ST STM32H743; Arrow (STM32U5); TechFeed (Pi 4). MCU power = current × 3.3 V [calculated]. Re-plotted by the team.

A ≈3-million-fold gap: this is why the **MCU** dominates battery-powered devices.
Price: **MCU** £9–12/chip · **SoC** $35/board · **FPGA** €144/chip [source; chip vs. board]

Our test case: one kernel, one specification, three platforms

Workload: a 16-tap low-pass FIR filter in Q15 fixed point

$$y[n] = \mathrm{sat}_{16}\Big(\big\lfloor \textstyle\sum_{k=0}^{15} h[k]\, x[n-k] \,/\, 2^{15} \big\rfloor \Big)$$

- Coefficients: windowed sinc (Hamming), $f_c = 0.2 f_s$, sum $= 2^{15}$ (unity DC gain)
- 1,024 input samples, generated once by a **Python golden model**
- The **same** coefficients and vectors are compiled into the MCU firmware, the Pi benchmark and the FPGA block RAM
- Pass criterion: **every output sample matches the golden model bit for bit**

Why bit-exact? If the three platforms computed slightly different things (float vs. fixed point, other rounding), a speed comparison would be unfair. One arithmetic specification removes that doubt.

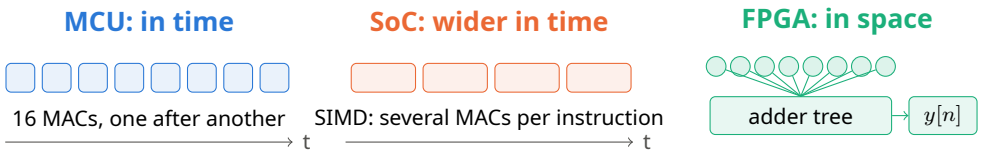
Status: FPGA RTL passes 1,024/1,024 samples in simulation, both with a continuous stream and with random gaps [simulated]

Test vectors: what each part checks

Section	Samples	What it checks
Unit impulse (32 767)	32	The output reproduces the 16 coefficients: catches tap-order or indexing errors
Step of 1,000	32	Settles at the DC gain: catches scaling and shift errors
Step at $+$max and $-$min	64	Full-scale inputs: catches sign and overflow handling
Adversarial pattern	32	Signs aligned with the coefficients, the largest possible sum: forces **saturation**
Alternating $\pm$max	32	Highest frequency at full scale: checks the stop band and sign flips
Uniform random (seed 3053)	832	The general case, and the bulk of the throughput measurement

60 of the 1,024 outputs saturate. Edge cases are tested on purpose, not only typical data.

Why FIR exposes the architecture



- FIR is pure **multiply-accumulate**: the building block of filtering, radio, beamforming and CNN convolution
- The same arithmetic maps to each architecture in its natural way, so the result reflects the **architecture**, not the algorithm
- A real filter is a **stream**: it tests sustained throughput *and* the time per sample, as a sensor or ADC would

What we measure, and how

Metric	**MCU** STM32F103	**FPGA** Arty Z7-20	**SoC** Raspberry Pi 4
Correctness	Compare to golden, 4 code versions	Self-check on chip, result over UART	Compare to golden, `-O2` and `-O3`
Throughput	`DWT->CYCCNT` cycles, interrupts off, median of 7	Cycle counter from first input to last output at 125 MHz	`clock_gettime`, pinned core, median of 15 runs of 2 M samples
Latency, determinism	Timer IRQ latency, 20,000 samples × 3 load scenarios	Pipeline depth in clock cycles	`cyclictest`, idle and under `stress-ng`
Power	Current meter: run, sleep, standby	Vivado `report_power` [tool estimate]	USB-C power meter, whole board
Resources	Flash, RAM	DSP, LUT, Fmax (Vivado)	—

Has anyone tested it this way before?

Prior work	What it compared	What it tells us
BDTI DSP Kernel Benchmarks	12 kernels, including real/complex block FIR, on DSP processors	FIR is an **industry-standard** kernel for comparing processors; BDTI notes kernels are hard to apply to FPGAs
EEMBC AutoBench	FIR on 16/32-bit fixed point, modelling an automotive/industrial task	Our 16-bit fixed-point FIR matches a **standard MCU workload**
Arucu & Iliev, 2025	FIR on Zynq FPGA vs. Tesla K80 GPU vs. Cortex-A9	FPGA ≈27× faster than the CPU, lowest power
Tatar et al., AICT 2022	FIR on ARM vs. Zynq FPGA+ARM vs. Intel i7	FPGA+ARM 8.86× faster than ARM alone
IEEE study (Xplore 10817876)	FIR of 3 lengths on Cyclone 10 LP, Stratix V, TI C6747 DSP, AMD Zen 3	Same FIR, several architectures: the method is accepted
MathWorks (STM32F746)	CMSIS-DSP vs. plain C on a Cortex-M7, cycle counts	Same MCU method as ours (cycles, CMSIS vs. C)

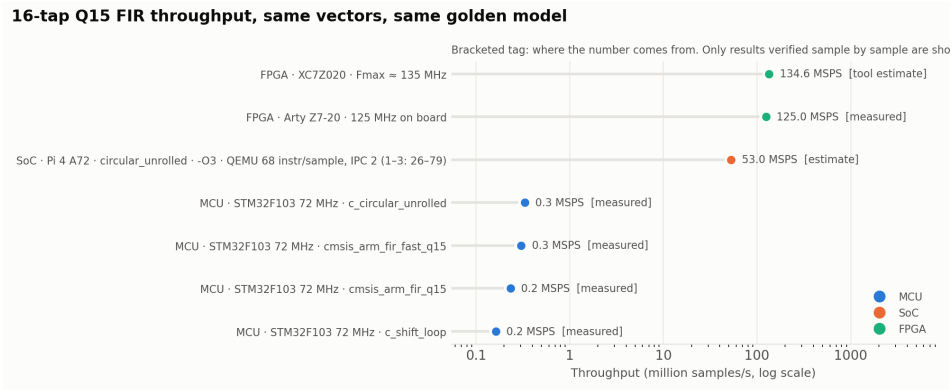
The gap we fill: none of these runs **one bit-exact FIR** on an **MCU**, an **FPGA** *and* a **SoC** with the same vectors.

Experiment setup

MCU	STM32F103, Cortex-M3	4 versions (plain C, unrolled C, CMSIS `arm_fir_q15`, `arm_fir_fast_q15`); interrupt latency in 3 scenarios
FPGA	Arty Z7-20 (XC7Z020 PL)	7-stage LUT-only pipeline (0 DSPs); self-check against golden on chip at 125 MHz; Vivado for Fmax and power
SoC	Raspberry Pi 4 (A72)	Same C code, `-O2` and `-O3` (NEON); `cyclictest`. Until the board is run: QEMU checks bit-exactness and counts instructions/sample

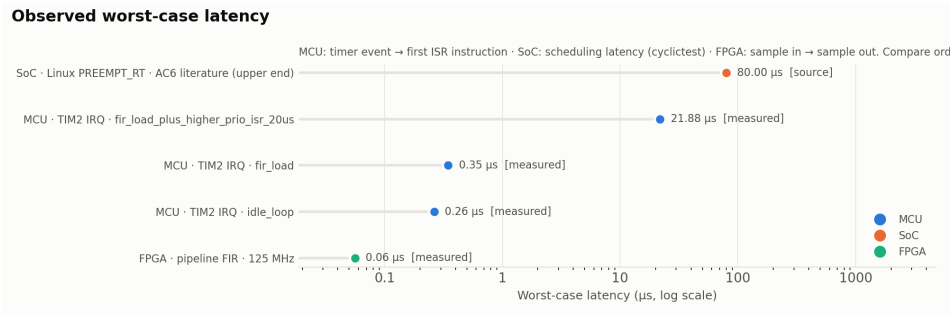
The Cortex-M3 has no SIMD instructions (SMLAD), so CMSIS-DSP gets no SIMD speed-up as it would on an M4

Results: throughput



MCU: 0.33 MSPS [measured] (**FPGA** is 375× faster) **FPGA**: 1.00 samples/cycle, 125.0 MSPS [measured] **SoC**: 53.0 MSPS [estimate; IPC 1–3: 26–79]

Results: worst-case latency



MCU IRQ while running FIR: 264–347 ns · with a higher-priority 20 µs ISR: 21.9 µs [measured]
FPGA pipeline: 7 cycles = 56 ns [measured] **SoC** standard Linux: a few ms [source: AC6] · PREEMPT_RT: 30–80 µs [source: AC6]

Cross-check against the literature

Question	We measured	Literature says
How close does the MCU get to “1 MAC per cycle”?	7% (216.3 cycles/sample)	The plan assumed 100%; flash needs 2 wait states at 72 MHz and `const` data is read from flash
Bare-metal MCU interrupt latency	264–347 ns	1–3 µs (AC6)
How much faster is the FPGA than the MCU / a CPU?	375× (vs. MCU); 2.4× (vs. Pi 4, estimate)	≈27× (vs. A9, Arucu & Iliev)
Does the FIR run bit-exact on ARMv8 (Cortex-A72)?	PASS (QEMU), 68 instr/sample	—
FPGA resources for the 16-tap FIR (LUT-only)	0/220 DSP · 1,672 LUT · 1,046 FF	Fmax ≈ 135 MHz, 0.124 W [tool estimate]

Side observation: one x86 core filters at ≈140 MSPS. The FPGA does not win on raw speed for small problems, but because **throughput stays constant as taps grow**, **latency is fixed** and **power is low**.

Summary: where each platform wins

	Typical domains	Deciding factor	Why not the other two
MCU	Automotive ECUs, motors, battery sensors	ASIL-D, 150 °C, nA idle current, a few dollars	SoC boots slowly and is not deterministic; FPGA is expensive
FPGA	Ultrasound, 5G radio, ASIC prototyping, HFT	Parallelism + direct I/O + reconfigurability	MCU lacks throughput; SoC is not deterministic
SoC	Wi-Fi routers, phones, IP cameras	Linux + hardware accelerators	MCU has no MMU; FPGA needs costly HDL work
Hybrid	ADAS, industrial control	OS + hard timing	No single platform covers both

Decision matrix

If the problem…	Choose	Because
Runs on a battery for years, light workload	**MCU**	300 nA in Standby
Has deadlines of tens of µs and needs safety certification	**MCU**	Sub-µs interrupts; ASIL-D
Needs over ≈0.5 G MAC/s sustained, or high-speed I/O	**FPGA** / hybrid	1 sample/clock, ≈44–48 G MAC/s
Needs nanosecond latency	**FPGA**	13.9 ns
Needs Linux, cameras, networking; soft timing	**SoC**	≈16× CoreMark; drivers ready-made
Needs both a rich OS and hard timing	Hybrid	Zynq, Lecture 3

Better timing and throughput are paid for with **higher cost** and **lower flexibility**

5
Conclusion

“There is no best platform,

only the one that is **good enough** for the deciding number of the problem,
at the lowest cost.”

— Nguyen Vo Hoang, Master’s student, NTU

Questions

Appendix: anticipated questions

- **“Is the MCU code optimized?”** Yes, there is a CMSIS-DSP version. The M3 has no SIMD; the best version reaches 216.3 cycles/sample.
- **“Is FPGA jitter really zero?”** RTL simulation has no real time. On silicon the latency is a fixed number of cycles; only ps-level clock jitter remains.
- **“Why compare a chip with a board?”** Only to compare orders of magnitude, and every table says so.
- **“How was FPGA power measured?”** It was not; it is Vivado `report_power` (vector-less) [tool estimate].
- **“Why no NRE figures?”** None are published. We use the time it took us to write the C and HDL versions as qualitative evidence.
- **“What about 64 taps?”** The MCU slows down about 4×. The FPGA still delivers 1 sample/clock; it only needs more LUTs/DSPs and a few more pipeline stages.

Main sources

- ST: STM32H7/F4/MP1 overview, STM32H743; OpenBenchmarking: Raspberry Pi 4
- Arucu & Iliev, J. Low Power Electron. Appl., 2025 (FIR on FPGA, GPU, CPU); Tatar et al., AICT 2022 (FIR on Zynq)
- BDTI DSP Kernel Benchmarks (bdti.com); EEMBC AutoBench FIR datasheet; IEEE Xplore 10817876 (FIR on FPGA vs. DSP); MathWorks CMSIS IFIR example on STM32F746
- AC6: Linux PREEMPT_RT latency; A-Team Insight: Exegy/AMD STAC-T0
- Embedded World 02/2026 (MCUs per car); Automotive World (AURIX TC3xx); Future Electronics (NXP S32K Grade 0)
- Macnica, CNX Software (Qualcomm IPQ); GSMArena (Snapdragon 8 Elite)
- Kou et al., 2023 (ultrasound beamforming on FPGA); HPCwire (Fujitsu + Xilinx O-RAN); Intel (Agilex O-RU); Synopsys HAPS
- eeNews Europe (Zynq UltraScale+); Lectures 2 and 3; Lee & Seshia §8.1

The full list with links is in the report (`report.pdf`).

Image credits

Photos: Wikimedia Commons, unmodified apart from scaling. Charts marked “Re-plotted by the team” were rebuilt from published data; no original figures were copied.

- Core Learning Board module Arduino STM32 F103 C8T6.jpg — Popolon, CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Core_Learning_Board_module_Arduino_STM32_F103_C8T6.jpg
- Raspberry Pi 4 Model B - Top.jpg — Laserlicht, CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Raspberry_Pi_4_Model_B_-_Top.jpg
- Xilinx Artix7 XC7A100T FGG484 on PCB.jpg — S03311251, CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Xilinx_Artix7_XC7A100T_FGG484_on_PCB.jpg
- FPGA cell example.png — Petter.kallstrom, Public domain. https://commons.wikimedia.org/wiki/File:FPGA_cell_example.png
- ECU and wire bundles.JPG — Snewkirk7953, CC BY-SA 3.0. https://commons.wikimedia.org/wiki/File:ECU_and_wire_bundles.JPG
- Intel FWIXP422BB on mainboard of UMTS Router Surf@home II, o2-8338.jpg — Raimond Spekking, CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Intel_FWIXP422BB_on_mainboard_of_UMTS_Router_Surf@home_II,_o2-8338.jpg
- HTC Desire - main board - Qualcomm QSD8250-2.jpg — Raimond Spekking, CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:HTC_Desire_-_main_board_-_Qualcomm_QSD8250-2.jpg
- Sonoscape Ultrasound Machine.jpg — Erereony, CC BY 4.0. https://commons.wikimedia.org/wiki/File:Sonoscape_Ultrasound_Machine.jpg
- Xilinx ZynqUltrascale XCZU4CG on PCB.jpg — S03311251, CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Xilinx_ZynqUltrascale_XCZU4CG_on_PCB.jpg